Lecture 21: MARS Simulator Setup & Your First MIPS Assembly Program

---

🎯 Learning Objectives

By the end of this lecture, students will be able to:

- Understand what the MARS Simulator is.
- Download and launch MARS.
- Explore the MARS interface.
- Understand the structure of a MIPS Assembly program.
- Learn the purpose of the `.data` and `.text` sections.
- Write, assemble, and execute their first MIPS program.
- Use comments and labels.
- Understand basic system calls for printing text and exiting a program.

---

1. Introduction

🎤 Teaching Script

First 30-Second SEO Phrase

Naturally mention:

> **"In today's Computer Organization and Assembly Language lecture, we'll install the MARS Simulator, explore its interface, and write our first MIPS Assembly program while learning the `.data` section, `.text` section, labels, comments, and basic system calls."**

In the previous lecture, we learned about the MIPS register set and how registers are used inside the processor.

Today, we'll begin writing actual MIPS Assembly programs.

To do that, we need a simulator because most of us don't have a real MIPS processor. We'll use **MARS (MIPS Assembler and Runtime Simulator)**, a free tool that allows us to write, assemble, execute, and debug MIPS programs.

By the end of this lecture, you'll have written and executed your very first MIPS Assembly program and understand the basic structure of every MIPS source file.

Let's get started.

---

2. What is MARS?

MARS stands for:

MIPS Assembler and Runtime Simulator

MARS is a software application that:

- Assembles MIPS Assembly code into machine code.
- Simulates a MIPS processor.
- Executes MIPS programs.
- Displays registers, memory, and program output.
- Supports debugging and step-by-step execution.

Why Use MARS?

- Free to use.
- Easy for beginners.
- No real MIPS hardware required.
- Visual interface for learning.
- Widely used in universities.

---

3. Installing MARS

Explain the installation process:

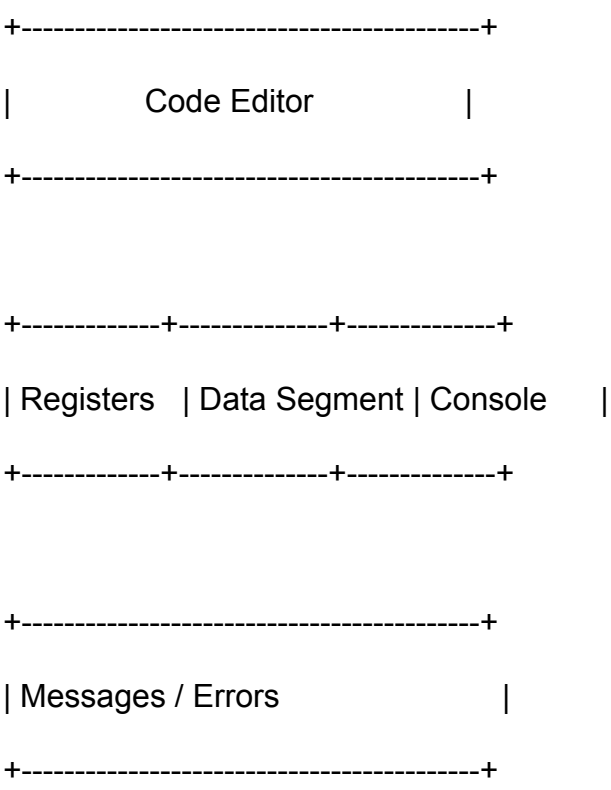
1. Download the latest MARS `.jar` file.
2. Ensure **Java** is installed on your computer.
3. Double-click the `.jar` file to launch MARS.

> **Instructor Tip:** Show students where to download MARS and verify that Java is installed before class.

---

4. MARS Interface Overview

When MARS opens, students will see several panels.



Explain each section:

Code Editor

- Write MIPS Assembly code.

Registers

- Displays values of all MIPS registers.

Data Segment

- Displays variables stored in memory.

Console

- Shows program output and accepts user input.

Messages

- Displays syntax errors and assembler messages.

---

5. Structure of a MIPS Program

Every MIPS program generally has two sections:

.data

Variables and strings

.text

Instructions

`.data`

Stores:

- Strings
- Variables

- Arrays
- Constants

`.text`

Contains executable instructions.

---

6. Comments in MIPS

Comments begin with the # symbol.

Example:

This program prints Hello World

Comments are ignored by the assembler and are used to explain the code.

Encourage students to comment every program, especially while learning.

---

7. Labels

Labels are used to identify locations in the code.

Example:

main:

Rules:

- End with a colon (`:`).
- No spaces.
- Used as jump targets.

8. First MIPS Program – Hello World

```
.data
message: .asciiz "Hello, World!"

.text
.globl main

main:

    # Print string
    li $v0, 4
    la $a0, message
    syscall

    # Exit program
    li $v0, 10
    syscall
```

9. Understanding the Program Line by Line

`.data`

.data

Starts the data section.

String Declaration

message: .asciiz "Hello, World!"

- `message` is a label.
- `.asciiz` stores a null-terminated string.

`.text`

.text

Starts the instruction section.

Global Main

.globl main

Makes the `main` label visible to the simulator as the program entry point.

Main Label

main:

Marks where program execution begins.

Load System Call Code

li $v0, 4

`4` tells the operating system:

> Print a string.

Load String Address

la $a0, message

Loads the address of the string into `$a0`.

Execute System Call

syscall

The CPU performs the requested service.

Output:

Hello, World!

Exit Program

li $v0, 10

syscall

System call `10` terminates the program.

---

10. Understanding System Calls

System calls allow MIPS programs to request services from the operating environment.

Code (`$v0`)	Operation
1	Print Integer
4	Print String
5	Read Integer
8	Read String
10	Exit Program

11	Print Character
12	Read Character

Mention that system calls will be covered in much greater detail in a dedicated lecture later.

---

11. How to Assemble and Run

Explain the workflow inside MARS:

1. Write the code.
2. Save the file (`.asm`).
3. Click **Assemble** (or press **F3**).
4. Fix any syntax errors if they appear.
5. Click **Run** (or press **F5**).
6. Observe the output in the Console.

---

12. Step-by-Step Debugging

Show students how to:

- Use **Step** to execute one instruction at a time.
- Watch register values change.
- Observe `$v0` and `$a0`.
- See how the Program Counter (PC) moves.

Explain that debugging is one of the best ways to understand Assembly Language.

---

13. Real-Life Analogy

Think of the MARS simulator as a virtual laboratory.

- Your code is the experiment.
- The assembler translates your instructions.
- The simulator behaves like a MIPS processor.
- Registers act as the CPU's working area.
- The console is the program's screen.

This lets you learn Assembly Language without needing physical MIPS hardware.

---

14. Common Mistakes

Students often:

❌ Forget the `.data` or `.text` section.

✔ Every program should clearly separate data and instructions.

❌ Forget the colon after a label.

Incorrect:

main

Correct:

main:

❌ Forget the `syscall` instruction.

Loading `$v0` alone does nothing until `syscall` is executed.

❌ Forget to exit the program.

Always finish with:

li $v0, 10

syscall

🎤 Lecture Summary

Today, we installed and explored the MARS Simulator and wrote our very first MIPS Assembly program.

We learned the structure of a MIPS source file, including the `.data` and `.text` sections, labels, comments, and basic system calls. We also assembled, executed, and debugged a simple program that printed a message to the console.

In the next lecture, we'll study **MIPS Data Types, Directives, and Memory Allocation**, where we'll learn how to declare integers, characters, strings, arrays, and reserve memory using assembler directives such as `.word`, `.byte`, `.half`, `.space`, and `.asciiz`.

Thank you for watching, and I'll see you in the next lecture.
